JavaScript is disabled on your browser.



compact1, compact2, compact3

java.util.concurrent.locks

# Class StampedLock

java.lang.Object
  java.util.concurrent.locks.StampedLock

**All Implemented Interfaces:**

Serializable

---

```
public class StampedLock
extends Object
implements Serializable
```

A capability-based lock with three modes for controlling read/write access. The state of a StampedLock consists of a version and mode. Lock acquisition methods return a stamp that represents and controls access with respect to a lock state; "try" versions of these methods may instead return the special value zero to represent failure to acquire access. Lock release and conversion methods require stamps as arguments, and fail if they do not match the state of the lock. The three modes are:

- **Writing.** Method `writeLock()` possibly blocks waiting for exclusive access, returning a stamp that can be used in method `unlockWrite(long)` to release the lock. Untimed and timed versions of `tryWriteLock` are also provided. When the lock is held in write mode, no read locks may be obtained, and all optimistic read validations will fail.
- **Reading.** Method `readLock()` possibly blocks waiting for non-exclusive access, returning a stamp that can be used in method `unlockRead(long)` to release the lock. Untimed and timed versions of `tryReadLock` are also provided.
- **Optimistic Reading.** Method `tryOptimisticRead()` returns a non-zero stamp only if the lock is not currently held in write mode. Method `validate(long)` returns true if the lock has not been acquired in write mode since obtaining a given stamp. This mode can be thought of as an extremely weak version of a read-lock, that can be broken by a writer at any time. The use of optimistic mode for short read-only code segments often reduces

contention and improves throughput. However, its use is inherently fragile. Optimistic read sections should only read fields and hold them in local variables for later use after validation. Fields read while in optimistic mode may be wildly inconsistent, so usage applies only when you are familiar enough with data representations to check consistency and/or repeatedly invoke method `validate()`. For example, such steps are typically required when first reading an object or array reference, and then accessing one of its fields, elements or methods.

This class also supports methods that conditionally provide conversions across the three modes. For example, method `tryConvertToWriteLock(long)` attempts to "upgrade" a mode, returning a valid write stamp if (1) already in writing mode (2) in reading mode and there are no other readers or (3) in optimistic mode and the lock is available. The forms of these methods are designed to help reduce some of the code bloat that otherwise occurs in retry-based designs.

StampedLocks are designed for use as internal utilities in the development of thread-safe components. Their use relies on knowledge of the internal properties of the data, objects, and methods they are protecting. They are not reentrant, so locked bodies should not call other unknown methods that may try to re-acquire locks (although you may pass a stamp to other methods that can use or convert it). The use of read lock modes relies on the associated code sections being side-effect-free. Unvalidated optimistic read sections cannot call methods that are not known to tolerate potential inconsistencies. Stamps use finite representations, and are not cryptographically secure (i.e., a valid stamp may be guessable). Stamp values may recycle after (no sooner than) one year of continuous operation. A stamp held without use or validation for longer than this period may fail to validate correctly. StampedLocks are serializable, but always deserialize into initial unlocked state, so they are not useful for remote locking.

The scheduling policy of StampedLock does not consistently prefer readers over writers or vice versa. All "try" methods are best-effort and do not necessarily conform to any scheduling or fairness policy. A zero return from any "try" method for acquiring or converting locks does not carry any information about the state of the lock; a subsequent invocation may succeed.

Because it supports coordinated usage across multiple lock modes, this class does not directly implement the `Lock` or `ReadWriteLock` interfaces. However, a StampedLock may be viewed `asReadLock()`, `asWriteLock()`, or `asReadWriteLock()` in applications requiring only the associated set of functionality.

**Sample Usage.** The following illustrates some usage idioms in a class that maintains simple two-dimensional points. The sample code illustrates some try/catch conventions even though they are not strictly needed here because no exceptions can occur in their bodies.

```
class Point {
  private double x, y;
  private final StampedLock sl = new StampedLock();
```

```
  void move(double deltaX, double deltaY) { // an exclusively locked method
    long stamp = sl.writeLock();
    try {
      x += deltaX;
      y += deltaY;
    } finally {
      sl.unlockWrite(stamp);
    }
  }

  double distanceFromOrigin() { // A read-only method
    long stamp = sl.tryOptimisticRead();
    double currentX = x, currentY = y;
    if (!sl.validate(stamp)) {
       stamp = sl.readLock();
       try {
         currentX = x;
         currentY = y;
       } finally {
          sl.unlockRead(stamp);
       }
    }
    return Math.sqrt(currentX * currentX + currentY * currentY);
  }

  void moveIfAtOrigin(double newX, double newY) { // upgrade
    // Could instead start with optimistic, not read mode
    long stamp = sl.readLock();
    try {
      while (x == 0.0 && y == 0.0) {
        long ws = sl.tryConvertToWriteLock(stamp);
        if (ws != 0L) {
          stamp = ws;
          x = newX;
          y = newY;
          break;
        }
        else {
          sl.unlockRead(stamp);
          stamp = sl.writeLock();
        }
      }
    } finally {
      sl.unlock(stamp);
    }
  }
}
```

**Since:**

1.8

**See Also:**

Serialized Form

## *Constructor Summary*

**Constructors**

| Constructor | Description |
| --- | --- |
| **StampedLock**() | Creates a new lock, initially in unlocked state. |

## *Method Summary*

**Concrete Methods**

| Modifier and Type | Method | Description |
| --- | --- | --- |
| **Lock** | **asReadLock**() | Returns a plain **Lock** view of this StampedLock in which the **Lock.lock()** method is mapped to **readLock()**, and similarly for other methods. |
| **ReadWriteLock** | **asReadWriteLock**() | Returns a **ReadWriteLock** view of this StampedLock in which the **ReadWriteLock.readLock()** method is mapped to **asReadLock()**, and **ReadWriteLock.writeLock()** to **asWriteLock()**. |
| **Lock** | **asWriteLock**() | Returns a plain **Lock** view of this StampedLock in which the **Lock.lock()** method is mapped to **writeLock()**, and similarly for other methods. |
| int | **getReadLockCount**() | Queries the number of read locks held for this lock. |
| boolean | **isReadLocked**() | Returns `true` if the lock is currently held non-exclusively. |
| boolean | **isWriteLocked**() | |

| | | |
|---|---|---|
| | | Returns `true` if the lock is currently held exclusively. |
| long | **readLock**() | Non-exclusively acquires the lock, blocking if necessary until available. |
| long | **readLockInterruptibly**() | Non-exclusively acquires the lock, blocking if necessary until available or the curren thread is interrupted. |
| **String** | **toString**() | Returns a string identifying this lock, as well as its lock state. |
| long | **tryConvertToOptimisticRead**(long stamp) | If the lock state matches the given stamp then, if the stamp represents holding a lock, releases it and returns an observation stamp. |
| long | **tryConvertToReadLock**(long stamp) | If the lock state matches the given stamp, performs one o the following actions. |
| long | **tryConvertToWriteLock**(long stamp) | If the lock state matches the given stamp, performs one o the following actions. |
| long | **tryOptimisticRead**() | Returns a stamp that can later be validated, or zero if exclusively locked. |
| long | **tryReadLock**() | Non-exclusively acquires the lock if it is immediately available. |
| long | **tryReadLock**(long time, **TimeUnit** unit) | Non-exclusively acquires the lock if it is available within the given time and the current thread has not been interrupted. |
| boolean | **tryUnlockRead**() | Releases one hold of the rea lock if it is held, without requiring a stamp value. |
| boolean | **tryUnlockWrite**() | |

| | | |
|---|---|---|
| | | Releases the write lock if it held, without requiring a stamp value. |
| long | **tryWriteLock**() | Exclusively acquires the lock if it is immediately available |
| long | **tryWriteLock**(long time, **TimeUnit** unit) | Exclusively acquires the lock if it is available within the given time and the current thread has not been interrupted. |
| void | **unlock**(long stamp) | If the lock state matches the given stamp, releases the corresponding mode of the lock. |
| void | **unlockRead**(long stamp) | If the lock state matches the given stamp, releases the non-exclusive lock. |
| void | **unlockWrite**(long stamp) | If the lock state matches the given stamp, releases the exclusive lock. |
| boolean | **validate**(long stamp) | Returns true if the lock has not been exclusively acquired since issuance of the given stamp. |
| long | **writeLock**() | Exclusively acquires the lock blocking if necessary until available. |
| long | **writeLockInterruptibly**() | Exclusively acquires the lock blocking if necessary until available or the current thread is interrupted. |

### Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

## *Constructor Detail*

### StampedLock

```
public StampedLock()
```

Creates a new lock, initially in unlocked state.

## *Method Detail*

### writeLock

```
public long writeLock()
```

Exclusively acquires the lock, blocking if necessary until available.

**Returns:**

a stamp that can be used to unlock or convert mode

### tryWriteLock

```
public long tryWriteLock()
```

Exclusively acquires the lock if it is immediately available.

**Returns:**

a stamp that can be used to unlock or convert mode, or zero if the lock is not available

### tryWriteLock

```
public long tryWriteLock(long time,
                         TimeUnit unit)
                  throws InterruptedException
```

Exclusively acquires the lock if it is available within the given time and the current thread has not been interrupted. Behavior under timeout and interruption matches that specified for method Lock.tryLock(long,TimeUnit).

**Parameters:**

time - the maximum time to wait for the lock

unit - the time unit of the time argument

**Returns:**

a stamp that can be used to unlock or convert mode, or zero if the lock is not available

**Throws:**

`InterruptedException` - if the current thread is interrupted before acquiring the lock

### writeLockInterruptibly

```
public long writeLockInterruptibly()
                            throws InterruptedException
```

Exclusively acquires the lock, blocking if necessary until available or the current thread is interrupted. Behavior under interruption matches that specified for method `Lock.lockInterruptibly()`.

**Returns:**

a stamp that can be used to unlock or convert mode

**Throws:**

`InterruptedException` - if the current thread is interrupted before acquiring the lock

### readLock

```
public long readLock()
```

Non-exclusively acquires the lock, blocking if necessary until available.

**Returns:**

a stamp that can be used to unlock or convert mode

### tryReadLock

```
public long tryReadLock()
```

Non-exclusively acquires the lock if it is immediately available.

**Returns:**

a stamp that can be used to unlock or convert mode, or zero if the lock is not available

### tryReadLock

```
public long tryReadLock(long time,
                        TimeUnit unit)
                 throws InterruptedException
```

Non-exclusively acquires the lock if it is available within the given time and the current thread has not been interrupted. Behavior under timeout and interruption matches that specified for method Lock.tryLock(long,TimeUnit).

**Parameters:**

time - the maximum time to wait for the lock

unit - the time unit of the time argument

**Returns:**

a stamp that can be used to unlock or convert mode, or zero if the lock is not available

**Throws:**

InterruptedException - if the current thread is interrupted before acquiring the lock

### readLockInterruptibly

```
public long readLockInterruptibly()
                            throws InterruptedException
```

Non-exclusively acquires the lock, blocking if necessary until available or the current thread is interrupted. Behavior under interruption matches that specified for method Lock.lockInterruptibly().

**Returns:**

a stamp that can be used to unlock or convert mode

**Throws:**

InterruptedException - if the current thread is interrupted before acquiring the lock

### tryOptimisticRead

```
public long tryOptimisticRead()
```

Returns a stamp that can later be validated, or zero if exclusively locked.

**Returns:**

a stamp, or zero if exclusively locked

### validate

```
public boolean validate(long stamp)
```

Returns true if the lock has not been exclusively acquired since issuance of the given stamp. Always returns false if the stamp is zero. Always returns true if the stamp represents a currently held lock. Invoking this method with a value not obtained from `tryOptimisticRead()` or a locking method for this lock has no defined effect or result.

**Parameters:**

`stamp - a stamp`

**Returns:**

`true if the lock has not been exclusively acquired since issuance of the given stamp; else false`

### unlockWrite

```
public void unlockWrite(long stamp)
```

If the lock state matches the given stamp, releases the exclusive lock.

**Parameters:**

`stamp - a stamp returned by a write-lock operation`

**Throws:**

`IllegalMonitorStateException - if the stamp does not match the current state of this lock`

### unlockRead

```
public void unlockRead(long stamp)
```

If the lock state matches the given stamp, releases the non-exclusive lock.

**Parameters:**

`stamp - a stamp returned by a read-lock operation`

**Throws:**

`IllegalMonitorStateException - if the stamp does not match the current state of this lock`

### unlock

```
public void unlock(long stamp)
```

If the lock state matches the given stamp, releases the corresponding mode of the lock.

**Parameters:**

`stamp - a stamp returned by a lock operation`

**Throws:**

`IllegalMonitorStateException - if the stamp does not match the current state of this lock`

### tryConvertToWriteLock

```
public long tryConvertToWriteLock(long stamp)
```

If the lock state matches the given stamp, performs one of the following actions. If the stamp represents holding a write lock, returns it. Or, if a read lock, if the write lock is available, releases the read lock and returns a write stamp. Or, if an optimistic read, returns a write stamp only if immediately available. This method returns zero in all other cases.

**Parameters:**

`stamp - a stamp`

**Returns:**

`a valid write stamp, or zero on failure`

### tryConvertToReadLock

```
public long tryConvertToReadLock(long stamp)
```

If the lock state matches the given stamp, performs one of the following actions. If the stamp represents holding a write lock, releases it and obtains a read lock. Or, if a read lock, returns it. Or, if an optimistic read, acquires a read lock and returns a read stamp only if immediately available. This method returns zero in all other cases.

**Parameters:**

`stamp - a stamp`

**Returns:**

`a valid read stamp, or zero on failure`

### tryConvertToOptimisticRead

```
public long tryConvertToOptimisticRead(long stamp)
```

If the lock state matches the given stamp then, if the stamp represents holding a lock, releases it and returns an observation stamp. Or, if an

optimistic read, returns it if validated. This method returns zero in all other cases, and so may be useful as a form of "tryUnlock".

**Parameters:**

`stamp - a stamp`

**Returns:**

`a valid optimistic read stamp, or zero on failure`

### tryUnlockWrite

```
public boolean tryUnlockWrite()
```

Releases the write lock if it is held, without requiring a stamp value. This method may be useful for recovery after errors.

**Returns:**

`true if the lock was held, else false`

### tryUnlockRead

```
public boolean tryUnlockRead()
```

Releases one hold of the read lock if it is held, without requiring a stamp value. This method may be useful for recovery after errors.

**Returns:**

`true if the read lock was held, else false`

### isWriteLocked

```
public boolean isWriteLocked()
```

Returns `true` if the lock is currently held exclusively.

**Returns:**

`true if the lock is currently held exclusively`

### isReadLocked

```
public boolean isReadLocked()
```

Returns `true` if the lock is currently held non-exclusively.

**Returns:**

`true if the lock is currently held non-exclusively`

### getReadLockCount

```
public int getReadLockCount()
```

Queries the number of read locks held for this lock. This method is designed for use in monitoring system state, not for synchronization control.

**Returns:**

the number of read locks held

### toString

```
public String toString()
```

Returns a string identifying this lock, as well as its lock state. The state, in brackets, includes the String "Unlocked" or the String "Write-locked" or the String "Read-locks:" followed by the current number of read-locks held.

**Overrides:**

toString in class Object

**Returns:**

a string identifying this lock, as well as its lock state

### asReadLock

```
public Lock asReadLock()
```

Returns a plain Lock view of this StampedLock in which the Lock.lock() method is mapped to readLock(), and similarly for other methods. The returned Lock does not support a Condition; method Lock.newCondition() throws UnsupportedOperationException.

**Returns:**

the lock

### asWriteLock

```
public Lock asWriteLock()
```

Returns a plain `Lock` view of this StampedLock in which the `Lock.lock()` method is mapped to `writeLock()`, and similarly for other methods. The returned Lock does not support a `Condition`; method `Lock.newCondition()` throws `UnsupportedOperationException`.

**Returns:**

the lock

### asReadWriteLock

```
public ReadWriteLock asReadWriteLock()
```

Returns a `ReadWriteLock` view of this StampedLock in which the `ReadWriteLock.readLock()` method is mapped to `asReadLock()`, and `ReadWriteLock.writeLock()` to `asWriteLock()`.

**Returns:**

the lock



Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.